

1. CMake와 GNU Make의 차이점

CMake는 프로그램을 직접 컴파일하는 도구가 아니라, 프로그램을 어떻게 빌드할 것인지 설정하고 Makefile과 같은 빌드 파일을 만들어주는 도구이다.

반면 GNU Make는 Makefile에 작성된 내용을 읽고 실제로 소스 코드를 컴파일하고 실행 파일을 만드는 빌드 도구이다.

쉽게 말하면 CMake는 "어떻게 빌드할지 준비해주는 도구"이고, GNU Make는 "준비된 내용을 보고 실제로 빌드하는 도구"라고 할 수 있다.

예제에서는 다음 명령어를 실행하면

```
`cmake -S src -B build`
```

CMake가 `CMakeLists.txt`를 읽고 `build` 디렉터리에 Makefile을 생성한다.

그 후 `build` 디렉터리에서

```
`make`
```

명령어를 실행하면 GNU Make가 Makefile을 읽어서 `main.cpp`를 컴파일하고 최종적으로 `Hello` 실행 파일을 생성한다.

2. CMakeLists.txt의 역할

`CMakeLists.txt`는 CMake가 프로젝트를 빌드하기 위해 필요한 정보를 작성하는 CMake의 설정 파일이다.

이 파일에는 프로젝트 이름, 사용할 소스 파일, 생성할 실행 파일이나 라이브러리 등의 정보를 작성한다.

예제에서는 다음과 같이 작성하였다.

```
`cmake_minimum_required(VERSION 3.16.3)`
```

```
`project(Hello)`
```

```
`add_executable(Hello main.cpp)`
```

각 명령어의 의미는 다음과 같다.

```
`cmake_minimum_required(VERSION 3.16.3)`
```

→ 사용할 CMake의 최소 버전을 지정한다.

```
`project(Hello)`
```

→ 프로젝트의 이름을 Hello로 지정한다.

```
`add_executable(Hello main.cpp)`
```

→ `main.cpp` 파일을 이용하여 `Hello`라는 실행 파일을 만들도록 설정한다.

즉, `CMakeLists.txt`는 CMake에게 "이 프로젝트를 어떤 방식으로 구성하고 빌드해야 하는지 알려주는 설명서" 역할을 한다.

3. CMakeCache.txt의 역할

`CMakeCache.txt`는 CMake의 Configure 과정에서 만들어지는 파일로, 프로젝트를 빌드하는 데 필요한 여러 설정값과 정보를 저장한다.

예를 들어 컴파일러의 위치, 링커의 위치, 빌드 프로그램의 위치, 빌드 옵션 등의 정보가 저장될 수 있다.

교수님 예제에서는 다음과 같은 정보를 확인할 수 있었다.

```
`CMAKE_C_COMPILER`
```

→ C 컴파일러의 위치

```
`CMAKE_CXX_COMPILER`
```

→ C++ 컴파일러의 위치

```
`CMAKE_LINKER`
```

→ 링커의 위치

```
`CMAKE_MAKE_PROGRAM`
```

→ Make 프로그램의 위치

예를 들어 다음과 같은 형태로 저장된다.

```
`CMAKE_CXX_COMPILER:FILEPATH=/usr/bin/c++`
```

```
`CMAKE_MAKE_PROGRAM:FILEPATH=/usr/bin/make`
```

CMake는 다음에 다시 Configure를 수행할 때 저장되어 있는 정보를 재사용할 수 있다.

따라서 `CMakeCache.txt`는 "CMake가 프로젝트를 설정하면서 확인한 빌드 환경과 설정값을 저장해 놓은 파일"이라고 할 수 있다.

4. CMake의 각 단계별 결과물 (Configure → Generate → Build)

CMake를 이용한 빌드 과정은 크게 Configure → Generate → Build의 순서로 진행된다.

① Configure 단계

Configure 단계에서는 CMake가 `CMakeLists.txt`를 읽고 프로젝트를 어떻게 빌드해야 하는지 확인한다.

또한 시스템에 설치된 컴파일러와 빌드 환경 등을 검사한다.

예를 들어 C++ 컴파일러가 어디에 있는지, 정상적으로 사용할 수 있는지 등을 확인한다.

이 과정에서 대표적으로 `CMakeCache.txt`와 `CMakeFiles` 디렉터리 등이 생성된다.

`CMakeCache.txt`에는 컴파일러 경로와 빌드 환경에 대한 설정값 등이 저장된다.

예제에서는 다음 명령어를 사용하였다.

```
`cmake -S src -B build`
```

정상적으로 Configure 과정이 완료되면 다음과 같은 메시지를 확인할 수 있다.

```
`-- Configuring done`
```

② Generate 단계

Generate 단계에서는 Configure 단계에서 확인한 정보를 바탕으로 실제 빌드 도구가 사용할 빌드 파일을 생성한다.

GNU Make를 사용하는 환경에서는 대표적으로 `Makefile`이 생성된다.

예제에서는 `hello/build/Makefile`이 생성된 것을 확인할 수 있었다.

Makefile에는 어떤 소스 파일을 컴파일해야 하는지, 어떤 순서로 빌드해야 하는지 등의 정보가 들어 있다.

정상적으로 Generate 과정이 끝나면 다음과 같은 메시지를 확인할 수 있다.

```
`-- Generating done`
```

즉, Configure 단계가 빌드 환경을 확인하고 설정하는 과정이라면, Generate 단계는 그 설정을 이용하여 실제 빌드 도구가 사용할 파일을 만드는 과정이다.

③ Build 단계

Build 단계에서는 Generate 단계에서 만들어진 빌드 파일을 이용하여 실제로 소스 코드를 컴파일하고 실행 파일을 생성한다.

이번 실습에서는 Generate 단계에서 `Makefile`이 생성되었기 때문에 `build` 디렉터리로 이동한 후 다음 명령어를 사용할 수 있다.

```
`make`
```

GNU Make는 Makefile을 읽고 `main.cpp`를 컴파일한 후 최종적으로 `Hello`라는 실행 파일을 생성한다.

빌드가 정상적으로 완료되면 다음과 같은 메시지를 확인할 수 있다.

```
`[100%] Built target Hello`
```

그리고

```
`./Hello`
```

명령어를 실행하면

```
`Hello World!`
```

가 출력된다.

정리

CMake의 전체 과정을 간단하게 정리하면 다음과 같다.

Configure

CMakeLists.txt 읽기 → 컴파일러와 빌드 환경 확인 → CMakeCache.txt 등의 설정 정보 생성

↓

Generate

Configure에서 확인한 정보를 이용 → Makefile과 같은 빌드 파일 생성

↓

Build

Makefile을 이용하여 실제 컴파일 및 링크 수행 → Hello 실행 파일 생성

따라서 CMake는 빌드에 필요한 환경을 확인하고 Makefile을 만들어주며, GNU Make는 생성된 Makefile을 이용하여 실제 프로그램을 빌드한다.

5.

```
linux@BOOK-M8LSJ7JPPD:~$ mkdir -p hello/src
linux@BOOK-M8LSJ7JPPD:~$ cd hello/src
linux@BOOK-M8LSJ7JPPD:~/hello/src$ pwd
/home/linux/hello/src
linux@BOOK-M8LSJ7JPPD:~/hello/src$ vi main.cpp
```

```
#include <iostream>

int main()
{
    std::cout << "Hello World!" << std::endl;
    return 0;
}
```

```
linux@BOOK-M8LSJ7JPPD:~/hello/src$ vi CMakeLists.txt
linux@BOOK-M8LSJ7JPPD:~/hello/src$
```

```
cmake_minimum_required(VERSION 3.16.3)
project>Hello)
add_executable>Hello main.cpp)
~
~
~
~
```

```
linux@BOOK-M8LSJ7JPPD:~/hello/src$ ls
CMakeLists.txt  main.cpp
```

```
linux@BOOK-M8LSJ7JPPD:~/hello/src$ cd ..
linux@BOOK-M8LSJ7JPPD:~/hello$ pwd
/home/linux/hello
```

```
linux@BOOK-M8LSJ7JPPD:~/hello$ cmake -S src -B build
-- Configuring done (0.0s)
-- Generating done (0.0s)
-- Build files have been written to: /home/linux/hello/build
```

```
linux@BOOK-M8LSJ7JPPD:~/hello$ ls
build  src
linux@BOOK-M8LSJ7JPPD:~/hello$ ls build
CMakeCache.txt  CMakeFiles  Hello  Makefile  cmake_install.cmake
```

```
linux@BOOK-M8LSJ7JPPD:~/hello$ cd build
linux@BOOK-M8LSJ7JPPD:~/hello/build$ pwd
/home/linux/hello/build
```

```
linux@BOOK-M8LSJ7JPPD:~/hello/build$ make
[ 50%] Building CXX object CMakeFiles>Hello.dir/main.cpp.o
[100%] Linking CXX executable Hello
[100%] Built target Hello
linux@BOOK-M8LSJ7JPPD:~/hello/build$
```

```
linux@BOOK-M8LSJ7JPPD:~/hello/build$ ls
CMakeCache.txt  CMakeFiles  Hello  Makefile  cmake_install.cmake
linux@BOOK-M8LSJ7JPPD:~/hello/build$ ./Hello
Hello World!
```